PatchOptic and Optic Category in `bai-harness`

Internal engineering report

Current codebase snapshot

Abstract

This report summarizes how `bai-harness` currently models PatchOptic-style enforcement and the newer Optic Category contracts. The key point is practical: permissions are declared and checked by deterministic code before effects run. The current system does not infer read or write authority from LLM text, does not run autonomous multi-agent loops, does not provide a hook registry, and does not sandbox trusted test commands.

1 Scope and Source of Truth

This report is based on the current repository implementation and tests: `README.md`, `docs/PLAN.md`, the `docs/dev/2026-05-13-patchoptic-composable-optics/` design notes, `src/bai/optic_category.py`, `src/bai/core/optics.py`, and the Optic Category e2e/dev tests.

Current implementation. The phase-one harness is a global CLI/runtime. Workspaces are explicitly registered under `BAI_HOME`; runtime artifacts live under `BAI_HOME/state`, `BAI_HOME/memory`, `BAI_HOME/config`, and related runtime directories. The system is serial by default.

Not implemented / future work. There is no hook registry, no PatchOptic runtime dependency, no autonomous agent loop, no agent-to-agent chat runtime, no general shell runner, and no workflow engine. The Optic Category layer is declarative validation and trace metadata, not an execution substrate.

Reader's Glossary

This report avoids assuming prior work on the development loop. The local terms mean:

PatchOptic	The harness's style of enforcing declared visibility, lineage, and write authority before effects run. It is not a third-party runtime dependency here.
Optic	A typed projection plus optional patch rules: what data is visible and what writes are allowed from that view.
Optic Category	The role-level layer that validates whether role optics compose through typed artifact handoffs.
CLI	Command-line interface, specifically the `bai` command.
MCP	Model Context Protocol. It is shown only as a conceptual tool-request input shape; this report does not claim an implemented MCP hook registry.
LLM	Large language model. User or model text does not grant permissions.
Lineage	Evidence showing which visible input or upstream artifact justifies an output or patch.
Write surface	The set of files or artifact locations a role is allowed to write.
BAI_HOME	The runtime home directory used by the harness. Runtime state, memory, approvals, and workflow artifacts are written there.
Harness	The serial owner of a request. It parses the command, owns the workspace registry, and routes proposed effects through policy checks.
PolicyEngine	Deterministic code that accepts or rejects inspect, test, artifact, and mutation effects before they run.
Workspace	A registered external project directory that the harness may inspect, test, or mutate only through approved paths.
Artifact	A durable JSON record under BAI_HOME, such as a workflow trace, audit record, approval record, or event.
argv	The exact command argument array approved for trusted test execution.
DAG	Directed acyclic graph: a dependency graph whose arrows do not loop back on themselves.
e2e test	End-to-end test that starts from the public command path or the closest public component boundary.

2 PatchOptic in This Harness

In this codebase, "PatchOptic" is best understood as an enforcement style: make visibility, lineage, and write surfaces explicit; validate them mechanically; and keep effects behind policy gates.

2.1 Declared Visibility

The field-level optics module, `src/bai/core/optics.py`, defines `ViewSpec`, `Optic`, `ProjectedView`, and `OpticPatch`. An optic has an input view, an output projection, and optional patch rules. A request may ask for a subset of declared output fields, but `ViewSpec.restrict()` rejects any field outside the declared projection.

The role-level category module, `src/bai/optic_category.py`, defines role optics with declared `input_types`, `output_types`, `read_projection`, `write_surface`, and `lineage_required`. For the bug-fix loop, request text such as "expose SecretTokenView" does not expand the trace input types or any role read projection.

2.2 Policy Gating

The runtime `PolicyEngine` sits on the effect path. It validates:

- agent output shape and serial execution policy;

- runtime artifact writes under owned `BAI_HOME` subtrees;
- inspect paths inside configured workspace allowed paths;
- test commands against explicitly approved argv or argv prefixes;
- code mutations only when an explicit Harness approval path is present.

Policy gates are not advisory. For example, mutation targets are checked before preimage inspection, mutation approvals, locks, or writes.

2.3 Lineage

Lineage appears at two levels:

- Field optics require explicit lineage tokens for visible fields and reject patch evidence that references non-visible lineage.
- Role optics declare required upstream artifact types. Composition validation fails if a role input or lineage requirement is not provided by an external composition input, a typed upstream handoff, or an explicit no-op marker where allowed.

2.4 Write Verification

Mutations remain Harness effects, not category-layer writes. A mutation approval artifact binds the exact workspace path, operation, content hash, task id, workspace id, approving mechanism, and expected preimage observed immediately before the write. Approved modify operations use atomic replacement guarded by a per-target runtime lock.

Role write surfaces are also validated declaratively. For example, `AuditorOptic` can write audit scripts, logs, and reports, but not production source or `docs/agents/`. `ReviewerOptic` can write `docs/agents/` only for the optional `AgentMapDelta` output.

2.5 Not LLM-Inferred Permissions

The implementation does not grant read or write permissions from user or model text. User text can request a task; it cannot expand `allowed_paths`, `read_projection`, declared view fields, role handoffs, or mutation approval scope. A mutation happens only through explicit approval and policy gating.

3 MCP / Tool Request vs PatchOptic Enforcement Stack

The diagram below includes MCP/tool requests as an input shape to explain the boundary. The current implementation uses the CLI/Harness path; it does not add a separate MCP runtime or hook registry. The important invariant is that any effect-like request must pass through the same deterministic gates before it touches workspace files, subprocess execution, memory, or runtime artifacts.

4 Workflow Codebooks: Useful but Limited

The source workflow skill notes are useful because they capture role intent: Auditor gathers evidence, Planner proposes a minimal repair plan, Patcher applies scoped changes, and Reviewer checks the result. They are good engineering documentation and a starting point for policy.

They are limited because prose does not enforce anything by itself. A codebook can say "Auditor must not modify production source," but enforcement requires deterministic validation at the runtime or

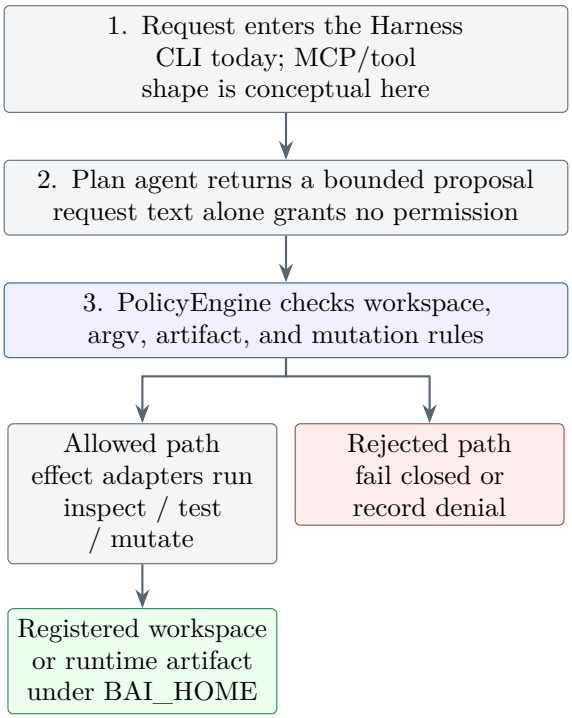

Figure 1: Current enforcement stack. MCP/tool requests are shown as a possible input form, not as an implemented registry.

declaration boundary. The Optic Category layer turns selected codebook constraints into structured declarations: inputs, outputs, read projections, write surfaces, typed handoffs, and lineage.

5 Simple Graph vs Optic Category

5.1 Simple Graph Wiring

A simple workflow graph records concrete execution order and dependencies. In `bai-harness`, the phase-one developer workflow artifact records `plan -> code -> doc -> test -> audit -> fix`. This is useful lifecycle metadata, but it is not enough to prove typed handoffs, bounded visibility, or role-specific write authority.

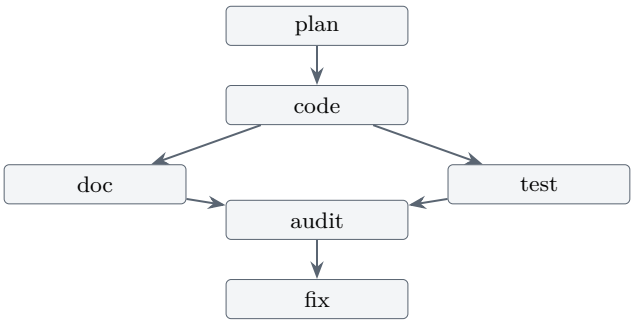

Figure 2: Simple graph: concrete artifact DAG for a developer workflow.

5.2 Optic Category Composition

The Optic Category layer validates whether role optics can compose. A handoff is valid only when a declared output type feeds the same declared input type of a later role. Composition validation also checks that required lineage is available, composition boundary inputs are not smuggled intermediate artifacts, unused undeclared visibility is rejected, and required/optional outputs are well formed.

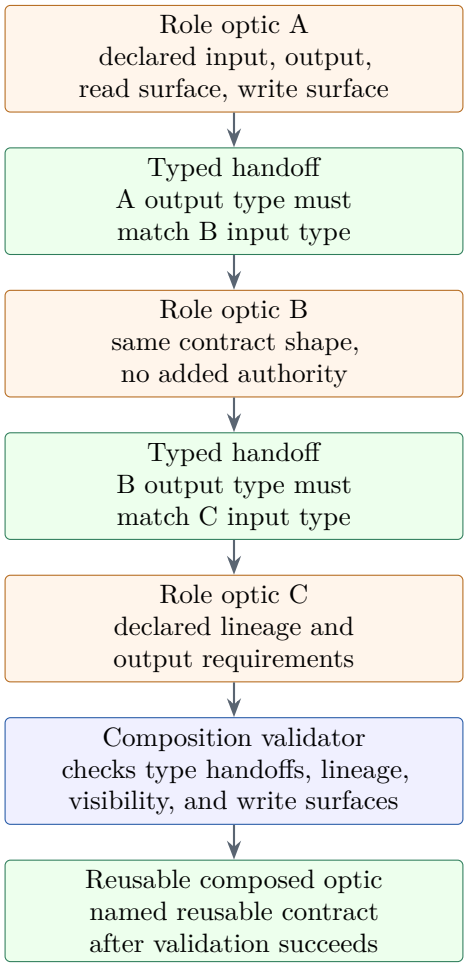

Figure 3: Optic Category: validated composition that can be referenced as a reusable contract.

6 Concrete Bug-Fix Workflow

The implemented bug-fix structural loop is `BugFixStructuralLoopOptic`. It is declared in the Optic Category source module and exercised by the dev/e2e tests listed in the source-of-truth section.

Read the roles below in plain English first. The exact path prefixes and type names are declared in the source module and checked by tests; the table avoids long path strings so the report stays readable.

Role optic	Inputs	Outputs	Write surface
Roadmap Extractor	Repository files	Repository map	Agent-map docs.
Auditor	RepositoryMapView, IssueClaimView	AuditReport	Audit scripts, logs, and reports; no production source writes.
Planner	RepositoryMapView, AuditReport	RepairPlan	Audit plan artifacts only; no agent-map docs.
Patcher	AuditReport, RepairPlan	PatchDiff	Repair-plan authorized files only; the symbolic placeholder is not a wildcard.
Reviewer	RepositoryMapView, AuditReport, RepairPlan, PatchDiff	ReviewReport, optional AgentMapDelta	Review artifacts; agent-map docs only when producing an AgentMapDelta.

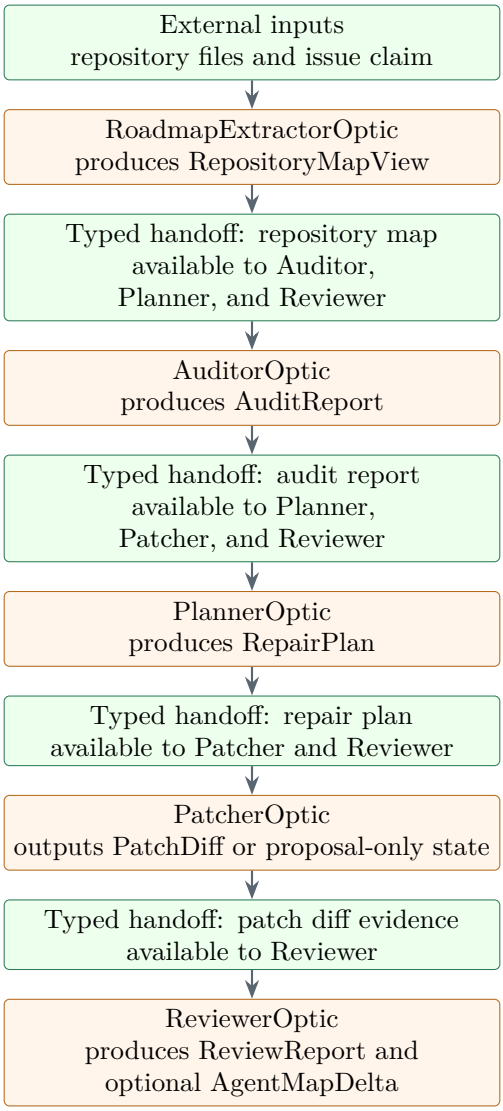

Figure 4: Bug-fix role workflow and typed handoffs.

7 E2E Example: `test_optic_category_bugfix_workflow.py`

The concrete stress test file is `tests/e2e/test_optic_category_bugfix_workflow.py`. It uses external `tmp_path` workspaces and isolated `BAI_HOME`. It starts from real `bai` CLI calls where appropriate.

7.1 Setup

One representative scenario creates a calculator workspace:

```
calc.py:
  def add(a, b):
      return a - b

test_calc.py:
  from calc import add
  def test_add():
      assert add(2, 3) == 5
```

The test registers the workspace with `bai workspace add demo <root>` and, for test-command paths, explicitly allows a pytest argv through `bai workspace allow-test`. The allowed test command uses `-B` and disables pytest cache in the stress test to avoid accidental workspace artifact writes.

7.2 CLI Requests

The structural trace case starts from:

```
bai run --workspace <workspace_id> "dev test <approved pytest argv>"
```

The approved mutation case intentionally does not rely on autonomous bug fixing. It uses the explicit current proposal shape:

```
bai run --workspace <workspace_id> \
  --approve-mutation <workspace>/calc.py \
  "dev propose-modify <workspace>/calc.py --content 'def add(a, b):
      return a + b
' test <approved pytest argv>"
```

The unapproved case omits `-approve-mutation`; the source file remains unchanged and the run records denied mutation evidence.

7.3 Expected Artifacts and Role Trace

The tests expect runtime artifacts under isolated `BAI_HOME`, not under the workspace or source checkout:

- workflow artifact under `state/workflows/`;
- context artifact under `state/context/`;
- audit artifact under `state/audits/`;
- optional fix artifact under `state/fixes/`;
- approval artifacts under `state/approvals/`;

- event artifacts under `state/events/`;
- optic trace artifact under `state/workflows/optic_traces/`.

The optic trace must contain `BugFixStructuralLoopOptic`, the role order `RoadmapExtractorOptic -> AuditorOptic -> PlannerOptic -> PatcherOptic -> ReviewerOptic`, exact typed handoffs, per-role input/output types, lineage, read projections, role statuses, and mutation evidence where a patch was explicitly approved and applied.

7.4 What This Proves

The e2e tests prove that the workflow is structured:

- the bug-fix loop has a reusable validated composition id;
- role order and typed handoffs are recorded;
- missing handoffs and undeclared visibility fail closed;
- role write surfaces are not broadened by composition;
- request text cannot expand read projections;
- approved mutation is bound to the exact path, content hash, expected preimage, task, workspace, and approval artifact;
- unapproved mutation remains proposal-only / denied.

The tests also prove what the workflow is *not*: it is not autonomous bug fixing. No patch is generated from the failing test. The only applied patch in the stress tests comes from an explicit `propose-modify` request plus `-approve-mutation`. Fix artifacts remain proposal-only.

8 Manual-Control E2E Coverage

`tests/e2e/test_scaffold_audit_manual_control.py` reinforces the same boundary from the CLI packaging path. It verifies that:

- `dev plan only` records plan, audit, workflow, and events without source mutation;
- unapproved fix proposals are audited and not applied;
- manual approval applies exactly one scoped change and records approval evidence;
- approvals for outside or unrequested paths do not authorize mutation;
- trusted test commands record `trusted_command_no_sandbox`;
- failing tests record failed audit/fix artifacts without success memory;
- background dev tasks are deferred metadata only.

9 Current Implementation vs Future Work

9.1 Current Implementation

- Deterministic PolicyEngine gates for runtime artifacts, inspect, trusted test command execution, and approved create/modify mutations.
- Field-level optics for declared projections and patch validation.
- Role-level Optic Category declarations for the bug-fix structural loop.
- Validation of typed handoffs, required lineage, optional outputs, invalid no-op inputs, undeclared visibility, and role write surfaces.
- Dev workflow artifacts and optic trace artifacts for phase-one runs.
- Explicit mutation approval artifacts with content hash and preimage binding.

9.2 Not Implemented

- No hook registry.
- No autonomous patch generation.
- No autonomous multi-agent loop or agent-to-agent chat runtime.
- No general shell runner.
- No filesystem sandbox for tests. Test commands are trusted direct execution and record `trusted_command_no_sandbox`.
- No cloud model routing or provider fallback in phase one.
- No benchmark/eval runner or self-improvement loop.

10 Engineering Takeaways

PatchOptic in this harness is not a magic permission system and not a category theory exercise. It is a set of enforceable contracts:

- declare what a role or optic can see;
- require lineage for what it writes;
- validate handoffs mechanically;
- keep actual effects behind Harness and PolicyEngine gates;
- record artifacts so reviewers can reconstruct what happened.

The Optic Category work makes workflow structure reusable without turning the harness into a workflow engine. The current bug-fix loop is a named validated contract that can be traced in e2e runs, while mutation authority remains explicit, scoped, and human-approved.